Classroom Chat Platform

Frontend Work Division Plan | 3-Member Development Team

1. Purpose of This Document

This document analyses the frontend requirements of the Classroom Chat Platform (React.js + Tailwind CSS) and divides the work across three frontend developers using an industry-standard, feature-ownership model. The split is designed so that each member owns a coherent vertical slice of the product (routes, components, state, and API integration), minimizing merge conflicts while enabling parallel development during the MVP phase.

2. Division Strategy

Work is split by **feature domain** rather than by page count, which is the standard approach used in professional React teams. Each member owns the full stack of a domain: UI components, routing, local state, API service calls, and WebSocket integration where relevant. A shared "Core/Design System" layer is built first (Week 1) so all three members can build independently afterward.

3. Team Roles at a Glance

Member	Ownership Area	Core Modules
Member 1	Core Platform & Auth Lead	Setup, Auth, Dashboard, Profile, Settings
Member 2	Classroom & Academics Lead	Classes, Classwork, Assignments, Grades
Member 3	Real-Time & Communication Lead	Chat, Notifications, WebSocket layer

4. Detailed Work Breakdown

Member 1 — Core Platform & Authentication Lead

Owns: project foundation, design system, authentication, dashboard, profile/settings

Owned Routes / Screens

Route	Screen	Notes
/, /login, /register	Landing, Login, Register	Public entry points, form validation
/forgot-password	Forgot / Reset Password	Token-based reset flow
/dashboard	Dashboard	Aggregated widgets: classes, deadlines, activity
/profile, /settings	Profile & Settings	Avatar upload, account preferences
/404	Not Found	Global fallback route

Key Components to Build

- Project scaffolding: Vite/CRA setup, folder structure, ESLint/Prettier, Tailwind config, theme tokens

- Axios instance with interceptors (JWT attach, refresh-token handling, global error toasts)
- AuthContext + ProtectedRoute / RoleBasedRoute wrapper (Teacher vs Student)
- Layout shell: Navbar, Sidebar, Footer, responsive AppShell used by every page
- Reusable common/ UI kit: Button, Input, Modal, Card, Badge, Loader, Toast, EmptyState
- Form components with validation (React Hook Form / Yup) for Login, Register, Reset Password

API / Service Integration

- POST /api/auth/register, /login, /refresh, /logout
- POST /api/auth/forgot-password, /reset-password
- GET /api/users/me, PUT /api/users/me (profile & settings)
- GET aggregated dashboard data (classes summary, upcoming deadlines)

Sprint Deliverables

- Week 1: Design system, routing skeleton, Axios/service layer (shared by all)
- Week 2: Auth flow fully working end-to-end (register → login → protected dashboard)
- Week 3: Dashboard widgets, Profile, Settings, responsive polish
- Week 4: Bug fixes, integration support for Members 2 & 3, final QA pass

Member 2 — Classroom & Academics Lead

Owns: classes, announcements, materials, assignments, submissions, grades

Owned Routes / Screens

Route	Screen	Notes
/classes, /classes/create, /classes/join	My Classes, Create, Join	Class code generation & join-by-code UX
/classes/:classId	Class Stream	Announcements feed
/classes/:classId/classwork	Classwork	Assignments + materials listing
/classes/:classId/materials	Materials	Study material upload/download
/classes/:classId/people	People	Teacher/Student roster, manage students
/classes/:classId/assignments/:assignmentId	Assignment Details, Submit, Review	Submission upload + teacher grading UI
(Grades view)	Grades	Per-student and per-class grade views

Key Components to Build

- ClassCard, ClassList, CreateClassForm, JoinClassForm (with class-code input)
- AnnouncementFeed + AnnouncementComposer (teacher only)
- MaterialList, MaterialUploadModal, FileAttachmentPreview
- AssignmentList, AssignmentCard, DeadlineBadge, AssignmentForm (create/edit, teacher)
- SubmissionUploader (student) and SubmissionReviewPanel (teacher: mark + feedback)
- PeopleList (roster) with role tags and remove/manage actions
- GradeTable / GradeSummary component, shared by student and teacher views

API / Service Integration

- GET/POST/PUT/DELETE /api/classes, /api/classes/{classId}/members
- GET/POST /api/classes/{classId}/announcements
- GET/POST /api/classes/{classId}/materials
- GET/POST/PUT /api/classes/{classId}/assignments, /api/assignments/{assignmentId}
- POST /api/submissions (file upload), PUT for teacher grading
- GET/POST /api/grades

Sprint Deliverables

- Week 1: Consume Member 1's layout/design system; build ClassList & Create/Join Class
- Week 2: Class Stream, Announcements, Materials
- Week 3: Assignments (create, list, submit) + file upload handling
- Week 4: Submission review, grading UI, Grades screen, responsive + edge cases

Member 3 — Real-Time & Communication Lead

Owns: WebSocket/STOMP layer, class & direct chat, notifications

Owned Routes / Screens

Route	Screen	Notes
/classes/:classId/chat	Class Group Chat	Real-time group messaging per class
/chat	Direct Chat / Chat Home	1:1 conversations, conversation list
/notifications	Notifications Center	Full notification history & filters
(global)	Notification Bell / Toast	Live in-app popups on every page

Key Components to Build

- WebSocket/STOMP service wrapper (connect, subscribe, reconnect logic) in services/websocket/
- ConversationList, ChatWindow, MessageBubble, MessageInput, FileAttachmentInChat
- TypingIndicator, OnlineStatusDot, ReadReceiptIndicator
- NotificationBell (navbar dropdown), NotificationList, NotificationItem, Toast integration
- useWebSocket / useChat / useNotifications custom hooks for reuse across pages

API / Service Integration

- GET/POST /api/conversations, GET/POST /api/messages (initial load + pagination)
- WebSocket topics: /topic/class/{classId}, /user/queue/messages, /user/queue/notifications
- GET/PUT /api/notifications (mark read/unread)
- Coordinates with Member 1 for JWT-authenticated STOMP handshake

Sprint Deliverables

- Week 1: WebSocket/STOMP client setup and connection handling (proof of concept)
- Week 2: Direct chat (1:1) fully functional with message history
- Week 3: Class group chat, typing indicators, online/offline status, file attachments
- Week 4: Notifications system end-to-end, cross-page toast integration, responsive + final polish

5. Shared Responsibilities

The following are owned collectively and reviewed by all three members, since they affect every module:

- Design tokens (colors, spacing, typography) defined once in Tailwind config — no hard-coded styles
- Responsive breakpoints (mobile / tablet / desktop) validated on every screen before merge
- Loading, empty, and error states — every list/detail view must handle all three
- Accessibility basics: semantic HTML, form labels, focus states, alt text
- Central error handling and toast/notification patterns (built by Member 1, used by all)

6. Industry-Standard Collaboration Workflow

6.1 Git Branching Strategy

The team follows a simplified **Git Flow**: **main** (production-ready), **develop** (integration branch), and short-lived **feature branches** per task, e.g. `feature/auth-login-page`, `feature/chat-websocket-setup`, `feature/assignment-submission`. No one commits directly to **main** or **develop**.

6.2 Commit Convention

Conventional Commits are used for a clean, readable history: `feat:`, `fix:`, `refactor:`, `style:`, `docs:`, `chore:` (e.g. `feat(chat): add typing indicator component`).

6.3 Code Review Process

- Every feature branch is merged into develop via a Pull Request — never a direct push
- At least one other team member reviews and approves before merge
- PRs stay small and scoped to a single screen/component where possible
- CI checks (lint, build) must pass before merge is allowed

6.4 Sprint Cadence

Work is organized into **four 1-week sprints** aligned to the tables above. Each sprint includes a short planning session at the start, daily async stand-up updates (what was done / next / blockers), and a demo + retro at the end of the week.

6.5 Interface Contracts

Before building, the team agrees on shared TypeScript/PropTypes interfaces for cross-cutting data (User, Class, Message, Notification) and mock API responses, so Members 2 and 3 can build UI against fake data while backend/API integration is finalized in parallel — avoiding blocking dependencies.

7. Dependencies Between Members

From → To	Dependency
Member 1 → Members 2 & 3	Design system, layout shell, Axios instance, AuthContext must land first (Week 1)
Member 1 → Member 3	JWT token access needed for authenticated STOMP/WebSocket handshake
Member 2 → Member 3	Class membership data (who belongs to a class) needed to scope class group chat

Member 3 → All	Notification toast component surfaces events triggered by Member 2's actions (new assignment, grade published)

8. Summary

This division gives each member full ownership of a vertical product slice, mirroring how professional frontend teams split React applications by domain rather than by individual pages. It keeps the MVP scope from the project README intact, minimizes cross-branch conflicts, and follows standard Git Flow, PR review, and sprint practices so the project is delivered as a cohesive, production-quality application rather than three disconnected pieces.